INSTITUT TEKNOLOGI DEL

MATERI TUGAS

Pemrograman Berorientasi Objek

SEMESTER GANJIL T.A 2026/2027

Tanggal / Sesi	:	Rabu, 7 Oktober 2026
Semester	:	Ganjil
Mata Kuliah	:	Pemrograman Berorientasi Objek
Minggu/Sesi	:	05/ 02
Aktivitas	:	Polymorphism & Abstract Class
Durasi	:	110 Menit
Setoran	:	Softfile
Batas Pengumpulan	:	Akhir Sesi
Tujuan	:	Mahasiswa mampu memahami konsep Polymorphism dan Abstract Class.

PENGANTAR STUDI KASUS

Perusahaan ekspedisi modern **LOGIX Express** menangani ribuan pengiriman setiap hari. Sistem perangkat lunak mereka dituntut memiliki fleksibilitas tinggi dan arsitektur yang aman dari manipulasi data.

LEVEL 1: MEMAHAMI POLIMORFISME

Polimorfisme berasal dari bahasa Yunani yang bermakna *"banyak bentuk"*. Dalam Java, konsep ini diwujudkan lewat method yang memiliki nama sama persis, tetapi menghasilkan perilaku atau variasi eksekusi yang berbeda. Polimorfisme terbagi menjadi dua ranah utama:

- **Compile-Time Polymorphism (Method Overloading):** Diputuskan saat kode dikompilasi, terjadi di dalam **satu kelas yang sama**.
- **Runtime Polymorphism (Method Overriding):** Diputuskan saat program berjalan, terjadi antar kelas dalam **hierarki pewarisan (parent-child)**.

Bagian 1.1: Method Overloading (Compile-Time Polymorphism)

Pada loket kasir LOGIX Express, petugas membutuhkan fungsi perhitungan tarif pengiriman (`hitungOngkir`) yang fleksibel. Pelanggan memiliki berbagai skenario pengiriman: hanya menghitung berdasarkan bobot dasar, menambah proteksi asuransi untuk barang bernilai tinggi, atau meminta pengiriman prioritas kilat.

Insight Konsep: Overloading (Beda Input, Beda Proses)

Bayangkan Anda sedang berada di kasir LOGIX Express. Anda bilang "Saya mau bayar". Kasir akan merespons berbeda tergantung apa yang Anda berikan:

- Jika Anda memberikan Uang Tunai ➔ Kasir menghitung kembalian.
- Jika Anda memberikan Kartu Debit ➔ Kasir meminta PIN.

Perintahnya sama-sama "Bayar" (nama method sama), tetapi proses di dalamnya berbeda karena input/parameternya berbeda. Inilah inti dari **Overloading**: *Java tahu harus menjalankan blok kode yang mana hanya dengan melihat apa yang dimasukkan ke dalam kurung parameter*.

Kode Program:

```
class MesinTarif {
    private String kotaAsal;

    public MesinTarif(String kotaAsal) {
        this.kotaAsal = kotaAsal;
    }

    public double hitungOngkir(double beratKg) {
        return beratKg * 10000;
    }

    public double hitungOngkir(double beratKg, double nilaiBarang) {
        double biayaAsuransi = nilaiBarang * 0.01;
        return (beratKg * 10000) + biayaAsuransi;
    }

    public double hitungOngkir(double beratKg, boolean isExpress) {
        double total = beratKg * 10000;
        if (isExpress) {
            total += 20000;
        }
        return total;
    }
}

public class SimulasiOverloading {
    public static void main(String[] args) {
        MesinTarif tarif = new MesinTarif("Medan");

        System.out.println("=== INI METHOD OVERLOADING ===");
        System.out.println("Ongkir Biasa (2.0 Kg)         : Rp " + tarif.hitungOngkir(2.0));
        System.out.println("Ongkir + Asuransi (Barang 5Jt): Rp " + tarif.hitungOngkir(2.0, 5000000.0));
        System.out.println("Ongkir Express (2.0 Kg)       : Rp " + tarif.hitungOngkir(2.0, true));
    }
}
```

Lembar Analisis Overloading:

- Mengapa compiler Java dapat menentukan method `hitungOngkir` mana yang dijalankan sebelum program dieksekusi (*compile-time*)? Bagian method apa yang diperiksa compiler?
- Jika pada class `MesinTarif` ditambahkan method berikut:

```
public int hitungOngkir(double beratKg) {

    return (int)(beratKg * 10000);

}
```

Mengapa compiler menolak kode tersebut padahal tipe kembaliannya berbeda (`int` vs `double`)?

Bagian 1.2: Method Overriding (Runtime Polymorphism)

Sistem LOGIX Express memiliki perintah penugasan standar: `kirimBarang(String alamat)`. Saat instruksi ini dipanggil pada kurir motor reguler, barang diantar melalui jalur darat. Namun, saat instruksi yang sama dijalankan pada kurir drone, barang diantar melintasi jalur udara.

Insight Konsep: Overriding (Satu Perintah, Beda Karakter)
Bayangkan Manajer Operasional berteriak melalui radio ke semua armada: "Kirim barang sekarang!" (nama method dan parameternya persis sama).

- Apa respons Kurir Motor? Dia menyalakan mesin, pakai helm, dan lewat jalan raya.
- Apa respons Kurir Drone? Dia memutar baling-baling dan terbang lurus di udara.

Inilah inti dari **Overriding**. *Perintah/instruksinya persis sama dari parent class, tetapi karena objek aslinya (child class) berbeda, maka cara mengeksekusinya (isi method-nya) disesuaikan dengan karakter masing-masing objek saat program berjalan (runtime).*

Aturan Teknis Overriding:

- Terjadi antar kelas yang memiliki relasi pewarisan (*inheritance*).
- Signature method (nama, tipe parameter, jumlah parameter) dan return type wajib persis sama (atau kompatibel kovarian).
- Method yang dieksekusi ditentukan saat program berjalan (*runtime*) oleh JVM berdasarkan objek aslinya, bukan berdasarkan tipe variabel referensinya.

Kode Program:

```
class KurirReguler {
    protected String nama;

    public KurirReguler(String nama) {
        this.nama = nama;
    }

    public void kirimBarang(String alamat) {
        System.out.println(nama + " mengantar paket ke " + alamat + " lewat jalur darat menembus kemacetan.");
    }
}

class KurirDrone extends KurirReguler {
    private String idDrone;

    public KurirDrone(String nama, String idDrone) {
        super(nama);
        this.idDrone = idDrone;
    }

    @Override
    public void kirimBarang(String alamat) {
        System.out.println("Unit Drone [" + idDrone + "] operator " + nama +
                           " terbang mengantar paket langsung ke koordinat: " + alamat);
    }
}

public class SimulasiOverriding {
    public static void main(String[] args) {
        System.out.println("=== INI METHOD OVERRIDING ===");

        KurirReguler k1 = new KurirReguler("Budi");

        KurirReguler k2 = new KurirDrone("Arka", "DRN-01");

        k1.kirimBarang("Jl. Sudirman No. 10");
        k2.kirimBarang("Gedung Graha Lt. 5");
    }
}
```

Lembar Analisis Overriding:

- Pada class `KurirDrone`, hapus atau jadikan komentar baris `@Override`. Kemudian, ubah nama method di bawahnya secara sengaja menjadi salah ketik, misalnya: `public void kirimbarang(String alamat)` (menggunakan huruf 'b' kecil).
- Jalankan `SimulasiOverriding.java`. Apa *output* yang tercetak dari `k2.kirimBarang("Gedung Graha Lt. 5")`? Apakah masih "lewat jalur udara" atau berubah menjadi "lewat jalur darat"?
- Sekarang kembalikan tulisan `@Override` di atas method yang salah ketik tersebut, lalu kompilasi ulang. Salin pesan *error* yang muncul!
- Berdasarkan eksperimen di atas, jelaskan dengan bahasa Anda sendiri apa fungsi utama dari tulisan `@Override` bagi seorang *programmer*!

Tantangan Ekstra – Polimorfisme

Buatlah sebuah **array bertipe parent class** (contoh: `KendaraanEkspedisi[] armada`) dan isi array tersebut dengan objek dari **child class yang berbeda**. Selanjutnya, gunakan **satu buah `for-loop`** untuk melakukan iterasi pada seluruh objek dalam array dan memanggil method `angkutBarang()`.

Pastikan implementasi memanfaatkan **polimorfisme**, sehingga method `angkutBarang()` dapat dipanggil melalui referensi parent class tanpa perlu membedakan jenis child class secara manual.

LEVEL 2: MEMAHAMI KELAS ABSTRAK

Abstraksi adalah prinsip menyembunyikan detail implementasi teknis dan hanya menetapkan fungsionalitasnya. Kelas abstrak (*abstract class*) bertindak sebagai cetak biru (*blueprint*) kerangka dasar yang belum lengkap, sehingga dilarang dibuat objek fisiknya secara langsung (`new`).

Bagian 2.1: Aturan & Karakteristik Kelas Abstrak

Staf gudang LOGIX Express secara keliru menulis kode `new KendaraanEkspedisi("B-00");`. Objek ini cacat logika karena tidak jelas jenis transportasinya: bukan motor, bukan mobil, dan bukan truk. Pihak arsitek sistem memutuskan mengubah `KendaraanEkspedisi` menjadi **Abstract Class**.

Insight Konsep: Abstraksi (Kerangka yang Belum Nyata)

Bayangkan Anda datang ke dealer otomotif dan berkata, "Pak, saya mau beli 1 unit Kendaraan Ekspedisi".

Dealer pasti bingung dan menolak pesanan Anda. Mengapa? Karena "Kendaraan Ekspedisi" hanyalah sebuah konsep (abstrak). Bentuk fisiknya belum jelas. Roda berapa? Pakai bensin atau listrik? Dealer baru bisa memproses jika Anda menyebutkan bentuk nyatanya (kelas konkret): "Pak, saya beli 1 unit Truk Box," atau "Saya beli 1 unit Mobil Van."

Di Java, kelas abstrak bertugas sebagai SOP (Standar Operasional) yang memaksa semua kendaraan ekspedisi untuk memiliki spesifikasi tertentu (misalnya wajib punya plat nomor dan fungsi `angkutBarang()`), tetapi Java melarang membuat objek nyatanya menggunakan kata kunci `new`.

Aturan Teknis:

- Tidak dapat diinstansiasi secara langsung (`new AbstractClass()` menghasilkan error).
- Dapat memiliki atribut dan constructor untuk diwariskan ke subclass via `super()`.
- Dapat memiliki concrete method (method biasa ber-body `{}`).
- Dapat memiliki abstract method (tanpa body `{}` dan diakhiri `;`), yang wajib diimplementasikan oleh subclass konkret.

Kode Program:

```
abstract class KendaraanEkspedisi {
    protected String noPolisi;
    protected int kapasitasKg;

    public KendaraanEkspedisi(String noPolisi, int kapasitasKg) {
        this.noPolisi = noPolisi;
        this.kapasitasKg = kapasitasKg;
    }

    public void infoKendaraan() {
        System.out.println("Plat Nomor: " + noPolisi + " | Daya Angkut: " + kapasitasKg + " Kg");
    }

    public abstract void angkutBarang(String namaBarang);
}

class TrukBox extends KendaraanEkspedisi {
    public TrukBox(String noPolisi, int kapasitasKg) {
        super(noPolisi, kapasitasKg);
    }

    @Override
    public void angkutBarang(String namaBarang) {
        System.out.println("Truk [" + noPolisi + "] memuat '" + namaBarang +
                           "' ke dalam kontainer tertutup pendingin.");
    }
}

class MobilVan extends KendaraanEkspedisi {
    public MobilVan(String noPolisi, int kapasitasKg) {
        super(noPolisi, kapasitasKg);
    }

    @Override
    public void angkutBarang(String namaBarang) {
        System.out.println("Mobil Van [" + noPolisi + "] memuat '" + namaBarang +
                           "' di bagasi belakang untuk distribusi kota.");
    }
}
```

```java
public class SimulasiAbstraksi {
    public static void main(String[] args) {
        System.out.println("=== INI ABSTRACT CLASS ===");

        // PERCOBAAN 1: Menguji larangan instansiasi langsung abstract class
        //KendaraanEkspedisi k = new KendaraanEkspedisi("BK 1234 DEL", 100);


        // PERCOBAAN 2: Instansiasi dari subclass konkret
        TrukBox fuso = new TrukBox("B 9081 TXT", 5000);
        fuso.infoKendaraan();
        fuso.angkutBarang("20 Unit Kulkas");

        System.out.println("--------------------------------");

        MobilVan grandmax = new MobilVan("B 1122 KAA", 800);
        grandmax.infoKendaraan();
        grandmax.angkutBarang("50 Kotak Sepatu");
    }
}
```

Lembar Analisis Abstraksi:

- **Eksperimen 1:** Pada class `SimulasiAbstraksi`, buka komentar (*uncomment*) pada baris PERCOBAAN 1: `KendaraanEkspedisi k = new KendaraanEkspedisi("BK 1234 DEL", 100);`
  Pertanyaan: Salin pesan *error* yang dihasilkan oleh Java. Mengacu pada "Kotak Insight Konsep" yang sudah Anda baca, mengapa Java menggagalkan baris kode ini?
- **Eksperimen 2:** Buatlah sebuah class baru dengan nama `MotorKurir` yang melakukan *extends* ke `KendaraanEkspedisi`, tetapi **jangan** menuliskan isi method `angkutBarang()` di dalamnya.
  Pertanyaan: Arahkan kursor (*hover*) ke garis merah *error* pada tulisan `class MotorKurir`. Tuliskan dua opsi solusi/perbaikan yang ditawarkan oleh IDE Java (seperti NetBeans/IntelliJ/Eclipse) untuk mengatasi *error* tersebut!

- Kumpulkan dalam bentuk folder zip dengan nama: **`PBO_W5S2_NIM.zip`**
- Isi folder: semua file `.java`
- **Berikan komentar pada setiap program** untuk menjelaskan baris code atau blok logika atau penjelasan fungsi program.